

Baa Corporate Bond Rate Analysis Report

Author: Thanh Vy Vo
Course: ISYE 6767 Fall 2025
Program: homework_1.cpp

1. Introduction

This report presents the implementation and experimental results of the C++ program designed to analyze historical annual yield rates of Moody-rated Baa seasoned corporate bonds. The program provides an interactive interface for querying bond rates by date and comparing them against historical averages.

2. Implementation

a. Program Architecture

The application follows a procedural design with three main components:

Data Management: The program uses two parallel vectors - `date` (strings) and `rate` (doubles) - to store the bond data, maintaining corresponding indices for date-rate pairs. This implementation preserves the relationship between dates and their corresponding rates.

Core Functions: Two primary functions (i.e., procedures) handle the computational logic:

- `average(vector<double> v)`: Computes the arithmetic mean of all bond rates in the dataset
- `find_rate(vector<double> rate_vec, vector<string> date_vec, string date)`: Performs linear search to locate a specific date and returns the corresponding rate, or -1 if not found

Interactive Interface: A continuous loop processes user input until EOF is encountered, allowing multiple queries in a single session.

b. Data Processing

The program reads from `hw1_H.15_Baa_Data.csv`, automatically skipping the first 6 header lines to focus on the actual data. Each line is parsed by splitting on the comma delimiter, with dates stored as strings in yyyy-mm format and rates converted from strings to double-precision numbers for subsequent mathematical operations (numerical comparisons and arithmetic mean).

c. Error Handling

The implementation includes error management for common failure scenarios:

- File access validation with appropriate error messages
- Invalid date format detection and user notification
- Missing date handling that allows continued program operation
- EOF detection for graceful program termination

3. Experimental Results

```
./hw1
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
2000-04
The annual rate for the input date is 8.4%
The annual rate for the input date is higher than the average rate by 1.3486%
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
2008-07
The annual rate for the input date is 7.16%
The annual rate for the input date is higher than the average rate by 0.108599%
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
1972-09
The annual rate for the input date is 8.09%
The annual rate for the input date is higher than the average rate by 1.0386%
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
```

Screenshot showing program execution with sample date queries, successful rate lookups, and comparison with average rates.

```
./hw1
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
200004
[Error] Date not found or the input is incorrectly formatted
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
June
[Error] Date not found or the input is incorrectly formatted
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
Hello
[Error] Date not found or the input is incorrectly formatted
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
2000-April
[Error] Date not found or the input is incorrectly formatted
Please input a date in yyyy-mm format, or Control-D (in Linux/Mac) or Control-Z (in windows) to end the program
```

Screenshot showing program execution with error handling for invalid inputs.

Through experimental testing, the program successfully performed:

- **Data Processing Capability:** The program loaded and processed the entire given CSV dataset, correctly calculating a historical average rate that serves as the baseline for all comparisons.

- **Query Accuracy:** Date lookups returned precise annual rates corresponding to the requested input date. The linear search algorithm successfully matched user inputs to the exact dates in the dataset.
- **Comparative Analysis:** For each valid query, the program accurately computed the difference between the specific date's rate and the historical average, providing insights into whether rates were above or below typical levels during specific periods.
- **Error Resilience:** Invalid date inputs and formatting errors were handled gracefully, enabling users to correct mistakes without needing to restart the program.

4. Results Summary

The Baa Corporate Bond Rate program successfully fulfilled its design objectives through experimental validation. The program, through repeated experiments, demonstrated reliable data processing, accurate rate retrieval, and correct statistical analysis. The interactive interface has clear output formatting and robust error handling that maintains program stability throughout testing.

The experimental results confirm that the implementation provides a practical tool for historical bond rate analysis, capable of handling data and user interactions effectively. The program's ability to process multiple queries in a single session while maintaining accuracy makes it suitable for comprehensive historical bond rate research.